

In reality, such an ideal combination of flexibility and blinding speed is generally not possible. Instead, game teams usually determine which types of queries are most likely to be needed during development of the game, and specialized data structures are implemented to accelerate those particular types of queries. As new queries become necessary, the engineers either leverage preexisting data structures to implement them, or they invent new ones if sufficient speed cannot be obtained. Here are a few examples of specialized data structures that can accelerate specific types of game object queries:

- *Finding game objects by unique id.* Pointers or handles to the game objects could be stored in a hash table or binary search tree keyed by unique id.
- *Iterating over all objects that meet a particular criterion.* The game objects could be presorted into linked lists based on various criteria (presuming the criteria are known a priori). For example, we might construct a list of all game objects of a particular type, maintain a list of all objects within a particular radius of the player, etc.
- *Finding all objects in the path of a projectile or with line of sight to some target point.* The collision system is usually leveraged to perform these kinds of game object queries. Most collision systems provide fast ray casts, and some also provide the ability to cast other shapes such as spheres or arbitrary convex volumes into the world to determine what they hit. (See Section 13.3.7.)
- *Finding all objects within a given region or radius.* We might consider storing our game objects in some kind of spatial hash data structure. This could be as simple as a horizontal grid placed over the entire game world or something more sophisticated, such as a quadtree, octree, kd-tree or other data structure that encodes spatial proximity.

## 16.6 Updating Game Objects in Real Time

Every game engine, from the simplest to the most complex, requires some means of updating the internal state of every game object over time. The *state* of a game object can be defined as the values of all its *attributes* (sometimes called its *properties*, and called *data members* in the C++ language). For example, the state of the ball in *Pong* is described by its  $(x, y)$  position on the screen and its velocity (speed and direction of travel). Because games are dynamic, time-based simulations, a game object's state describes its configuration at *one specific instant in time*. In other words, a game object's notion of time is *discrete* rather than *continuous*. (However, as we'll see, it's helpful to think of the ob-

jects' states as changing continuously and then being sampled discretely by the engine, because it helps you to avoid some common pitfalls.)

In the following discussions, we'll use the symbol  $\mathbf{S}_i(t)$  to denote the state of object  $i$  at an arbitrary time  $t$ . The use of vector notation here is not strictly mathematically correct, but it reminds us that a game object's state acts like a heterogeneous  $n$ -dimensional vector, containing all sorts of information of various data types. We should note that this usage of the term "state" is not the same as the states in a *finite state machine*. A game object may very well be implemented in terms of one—or many—finite state machines, but in that case, a specification of the current state of each FSM would merely be a part of the game object's overall state vector  $\mathbf{S}(t)$ .

Most low-level engine subsystems (rendering, animation, collision, physics, audio and so on) require periodic updating, and the game object system is no exception. As we saw in Chapter 8, updating is usually done via a single master loop called the *game loop*. Virtually all game engines update game object states as part of their main game loop—in other words, they treat the game object model as just another engine subsystem that requires periodic servicing.

Game object updating can therefore be thought of as the process of determining the state of each object at the current time  $\mathbf{S}_i(t)$  given its state at a previous time  $\mathbf{S}_i(t - \Delta t)$ . Once all object states have been updated, the current time  $t$  becomes the new previous time  $(t - \Delta t)$ , and this process repeats for as long as the game is running. Usually, one or more *clocks* are maintained by the engine—one that tracks real time exactly and possibly others that may or may not correspond to real time. These clocks provide the engine with the absolute time  $t$  and/or with the change in time  $\Delta t$  from iteration to iteration of the game loop. The clock that drives the updating of game object states is usually permitted to diverge from real time. This allows the behaviors of the game objects to be paused, slowed down, sped up or even run in reverse—whatever is required in order to suit the needs of the game design. These features are also invaluable for debugging and development of the game.

As we learned in Chapter 1, a game object updating system is an example of what is known as a *dynamic, real-time, agent-based computer simulation* in computer science. Game object updating systems also exhibit some aspects of *discrete event simulations* (see Section 16.8 for more details on events). These are well-researched areas of computer science, and they have many applications outside the field of interactive entertainment. Games are one of the more complex kinds of agent-based simulation—as we'll see, updating game object states over time in a dynamic, interactive virtual environment can be surprisingly difficult to get right. Game programmers can learn a lot about game

object updating by studying the wider field of agent-based and discrete event simulations. And researchers in those fields can probably learn a thing or two from game engine design as well!

As with all high-level game engine systems, every engine takes a slightly (or sometimes radically) different approach. However, as before, most game teams encounter a common set of problems, and certain design patterns tend to crop up again and again in virtually every engine. In this section, we'll investigate these common problems and some common solutions to them. Please bear in mind that game engines may exist that employ very different solutions to the ones described here, and some game designs face unique problems that we can't possibly cover here.

### 16.6.1 A Simple Approach (That Doesn't Work)

The simplest way to update the states of a collection of game objects is to iterate over the collection and call a virtual function, named something like `Update()`, on each object in turn. This is typically done once during each iteration of the main game loop (i.e., once per *frame*). Game object classes can provide custom implementations of the `Update()` function in order to perform whatever tasks are required to advance the state of that type of object to the next discrete time index. The time delta from the previous frame can be passed to the update function so that objects can take proper account of the passage of time. At its simplest, then, our `Update()` function's signature might look something like this:

```
virtual void Update(float dt);
```

For the purposes of the following discussions, we'll assume that our engine employs a monolithic object hierarchy, in which each game object is represented by a single instance of a single class. However, we can easily extend the ideas here to virtually any object-centric design. For example, to update a component-based object model, we could call `Update()` on every component that makes up each game object, or we could call `Update()` on the "hub" object and let it update its associated components as it sees fit. We can also extend these ideas to property-centric designs, by calling some sort of `Update()` function on each property instance every frame.

They say that the devil is in the details, so let's investigate two important details here. First, how should we maintain the collection of all game objects? And second, what kinds of things should the `Update()` function be responsible for doing?

### 16.6.1.1 Maintaining a Collection of Active Game Objects

The collection of active game objects is often maintained by a singleton manager class, perhaps named something like `GameWorld` or `GameObjectManager`. The collection of game objects generally needs to be dynamic, because game objects are spawned and destroyed as the game is played. Hence a *linked list* of pointers, smart pointers or handles to game objects is one simple and effective approach. (Some game engines disallow dynamic spawning and destroying of game objects; such engines can use a statically sized *array* of game object pointers, smart pointers or handles rather than a linked list.) As we'll see below, most engines use more complex data structures to keep track of their game objects rather than just a simple, flat linked list. But for the time being, we can visualize the data structure as a linked list for simplicity.

### 16.6.1.2 Responsibilities of the `Update()` Function

A game object's `Update()` function is primarily responsible for determining the state of that game object at the current discrete time index  $S_i(t)$  given its previous state  $S_i(t - \Delta t)$ . Doing this may involve applying a rigid body dynamics simulation to the object, sampling a preauthored animation, reacting to events that have occurred during the current time step and so on.

Most game objects interact with one or more engine subsystems. They may need to animate, be rendered, emit particle effects, play audio, collide with other objects and static geometry and so on. Each of these systems has an internal state that must also be updated over time, usually once or a few times per frame. It might seem reasonable and intuitive to simply update all of these subsystems directly from within the game object's `Update()` function. For example, consider the following hypothetical update function for a Tank object:

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Now update low-level engine subsystems on behalf
    // of this tank. (NOT a good idea... see below!)
    m_pAnimationComponent->Update(dt);
    m_pCollisionComponent->Update(dt);
    m_pPhysicsComponent->Update(dt);
    m_pAudioComponent->Update(dt);
    m_pRenderingComponent->draw();
}
```



```
}
```

Given that our `Update()` functions are structured like this, the game loop could be driven almost entirely by the updating of the game objects, like this:

```
while (true)
{
    PollJoypad();

    float dt = g_gameClock.CalculateDeltaTime();

    for (each gameObject)
    {
        // This hypothetical Update() function updates
        // all engine subsystems!
        gameObject.Update(dt);
    }

    g_renderingEngine.SwapBuffers();
}
```

However attractive the simple approach to object updating shown above may seem, it is usually not viable in a commercial-grade game engine. In the following sections, we'll explore some of the problems with this simplistic approach and investigate common ways in which each problem can be solved.

### 16.6.2 Performance Constraints and Batched Updates

Most low-level engine systems have extremely stringent performance constraints. They operate on a large quantity of data, and they must do a large number of calculations every frame as quickly as possible. As a result, most engine systems benefit from *batched updating*. For example, it is usually far more efficient to update a large number of animations in one batch than it is to update each object's animation interleaved with other unrelated operations, such as collision detection, physical simulation and rendering.

In most commercial game engines, each engine subsystem is updated directly or indirectly by the main game loop rather than being updated on a per-game object basis from within each object's `Update()` function. If a game object requires the services of a particular engine subsystem, it asks that subsystem to allocate some subsystem-specific state information on its behalf. For example, a game object that wishes to be rendered via a triangle mesh might request the rendering subsystem to allocate a *mesh instance* for its use. (As described in Section 11.1.1.5, a mesh instance represents a single instance of a

triangle mesh—it keeps track of the position, orientation and scale of the instance in world space whether or not it is visible, per-instance material data and any other per-instance information that may be relevant.) The rendering engine maintains a collection of mesh instances internally. It can manage the mesh instances however it sees fit in order to maximize its own runtime performance. The game object controls *how* it is rendered by manipulating the properties of the mesh instance object, but the game object does not control the rendering of the mesh instance directly. Instead, after all game objects have had a chance to update themselves, the rendering engine draws all visible mesh instances in one efficient batch update.

With batched updating, a particular game object's `Update()` function, such as that of our hypothetical tank object, might look more like this:

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Control the properties of my various engine
    // subsystem components, but do NOT update
    // them here...

    if (justExploded)
    {
        m_pAnimationComponent->PlayAnimation("explode");
    }

    if (isVisible)
    {
        m_pCollisionComponent->Activate();
        m_pRenderingComponent->Show();
    }
    else
    {
        m_pCollisionComponent->Deactivate();
        m_pRenderingComponent->Hide();
    }

    // etc.
}
```

The game loop then ends up looking more like this:

```
while (true)
{
```

```
PollJoypad();

float dt = g_gameClock.CalculateDeltaTime();

for (each gameObject)
{
    gameObject.Update(dt);
}

g_animationEngine.Update(dt);
g_physicsEngine.Simulate(dt);
g_collisionEngine.DetectAndResolveCollisions(dt);
g_audioEngine.Update(dt);
g_renderingEngine.RenderFrameAndSwapBuffers();
}
```

Batched updating provides many performance benefits, including but not limited to:

- *Maximal cache coherency.* Batched updating allows an engine subsystem to achieve maximum cache coherency because its per-object data is maintained internally and can be arranged in a single, contiguous region of RAM.
- *Minimal duplication of computations.* Global calculations can be done once and reused for many game objects rather than being redone for each object.
- *Reduced reallocation of resources.* Engine subsystems often need to allocate and manage memory and/or other resources during their updates. If the update of a particular subsystem is interleaved with those of other engine subsystems, these resources must be freed and reallocated for each game object that is processed. But if the updates are batched, the resources can be allocated once per frame and reused for all objects in the batch.
- *Efficient pipelining.* Many engine subsystems perform a virtually identical set of calculations on each and every object in the game world. When updates are batched, a scatter/gather approach can be employed to divide large workloads across multiple CPU cores. This kind of parallelism cannot be achieved when processing each object in isolation.

Performance benefits aren't the only reason to favor a batch updating approach. Some engine subsystems simply don't work at all when updated on a per-object basis. For example, if we are trying to resolve collisions within a system of multiple dynamic rigid bodies, a satisfactory solution cannot be

found in general by considering each object in isolation. The interpenetrations between these objects must be resolved as a group, either via an iterative approach or by solving a linear system.

### 16.6.3 Object and Subsystem Interdependencies

Even if we didn't care about performance, a simplistic per-object updating approach breaks down when game objects *depend* on one another. For example, a human character might be holding a cat in her arms. In order to calculate the world-space pose of the cat's skeleton, we first need to calculate the world-space pose of the human. This implies that the *order* in which objects are updated is important to the proper functioning of the game.

Another related problem arises when engine *subsystems* depend on one another. For example, a rag doll physics simulation must be updated in concert with the animation engine. Typically, the animation system produces an intermediate, local-space skeletal pose. These joint transforms are converted to world space and applied to a system of connected rigid bodies that approximate the skeleton within the physics system. The rigid bodies are simulated forward in time by the physics system, and then the final resting places of the joints are applied back to their corresponding joints in the skeleton. Finally, the animation system calculates the world-space pose and skinning matrix palette. So once again, the updating of the animation and physics systems must occur in a particular order in order to produce correct results. These kinds of inter-subsystem dependencies are commonplace in game engine design.

#### 16.6.3.1 Phased Updates

To account for inter-subsystem dependencies, we can explicitly code our engine subsystem updates in the proper order within the main game loop. For example, to handle the interplay between the animation system and rag doll physics, we might write something like this:

```
while (true) // main game loop
{
    // ...

    g_animationEngine.CalculateIntermediatePoses(dt);
    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
    g_animationEngine.FinalizePoseAndMatrixPalette();
}
```

```

        // ...
    }

```

We must be careful to update the states of our game objects at the right time during the game loop. This is often not as simple as calling a single `Update()` function per game object per frame. Game objects may depend upon the intermediate results of calculations performed by various engine subsystems. For example, a game object might request that animations be played prior to the animation system running its update. However, that same object may also want to procedurally adjust the intermediate pose generated by the animation system prior to that pose being used by the rag doll physics system and/or the final pose and matrix palette being generated. This implies that the object must be updated twice, once before the animation calculates its intermediate poses and once afterward.

Many game engines allow their game objects to run update logic at multiple points during the frame. For example, the Naughty Dog engine (the engine that drives the *Uncharted* and *The Last of Us* game series) updates game objects three times—once before animation blending, once after animation blending but prior to final pose generation and once after final pose generation. This can be accomplished by providing each game object class with three virtual functions that act as “hooks.” In such a system, the game loop ends up looking something like this:

```

while (true) // main game loop
{
    // ...

    for (each gameObject)
    {
        gameObject.PreAnimUpdate(dt);
    }

    g_animationEngine.CalculateIntermediatePoses(dt);

    for (each gameObject)
    {
        gameObject.PostAnimUpdate(dt);
    }

    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
}

```

```
g_animationEngine.FinalizePoseAndMatrixPalette();

for (each gameObject)
{
    gameObject.FinalUpdate(dt);
}

// ...
}
```

We can provide our game objects with as many update phases as we see fit. But we must be careful, because iterating over all game objects and calling a virtual function on each one can be expensive. Also, not all game objects require all update phases—iterating over objects that don’t require a particular phase is a pure waste of CPU bandwidth.

Actually, the above example isn’t completely realistic. Iterating directly over all game objects to call their `PreAnimUpdate()`, `PostAnimUpdate()` and `FinalUpdate()` hook functions would be highly inefficient, because only a small percentage of the objects might actually need to perform any logic in each hook. It’s also an inflexible design, because only game objects are supported—if we wanted to update a particle system during the post-animation phase, we’d be out of luck. Finally, such a design would lead to unnecessary *coupling* between the low-level engine systems and the game object system.

A generic callback mechanism would be a much better design choice. In such a design, the animation system would provide a facility by which any client code (game objects or any other engine system) could *register a callback function* for each of the three update phases (pre-animation, post-animation and final). The animation system would iterate through all registered callbacks and call them, without any “knowledge” of game objects *per se*. This design maximizes performance, because only those clients that actually *need* updates register callbacks and are called each frame. It also maximizes flexibility and eliminates unnecessary coupling between the game object system and other engine subsystems, because any client is allowed to register a callback, not just game objects.

#### 16.6.3.2 Bucketed Updates

In the presence of *inter-object* dependencies, the phased updates technique described above must be adjusted a little. This is because inter-object dependencies can lead to conflicting rules governing the order of updating. For example, let’s imagine that object B is being held by object A. Further, let’s assume that we can only update object B after A has been *fully* updated, including the

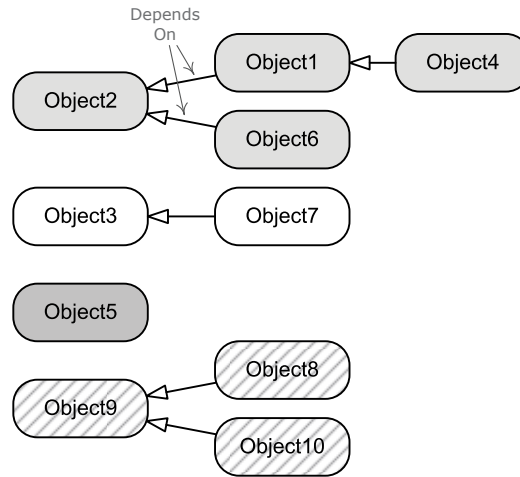


Figure 16.14. Inter-object update order dependencies can be viewed as a forest of dependency trees.

calculation of its final world-space pose and matrix palette. This conflicts with the need to batch animation updates of all game objects together in order to allow the animation system to achieve maximum throughput.

Inter-object dependencies can be visualized as a forest of dependency trees. The game objects with no parents (no dependencies on any other object) represent the roots of the forest. An object that depends directly on one of these root objects resides in the first tier of children in one of the trees in the forest. An object that depends on a first-tier child becomes a second-tier child and so on. This is illustrated in Figure 16.14.

One solution to the problem of conflicting update order requirements is to collect objects into independent groups, which we'll call *buckets* here for lack of a better name. The first bucket consists of all root objects in the forest. The second bucket is comprised of all first-tier children. The third bucket contains all second-tier children and so on. For each bucket, we run a complete update of the game objects and the engine systems, complete with all update phases. Then we repeat the entire process for each bucket until there are no more buckets.

In theory, the depths of the trees in our dependency forest are unbounded. However, in practice, they are usually quite shallow. For example, we might have characters holding weapons, and those characters might or might not be riding on a moving platform or a vehicle. To implement this, we only need three tiers in our dependency forest, and hence only three buckets: one for

platforms/vehicles, one for characters and one for the weapons in the characters' hands. Many game engines explicitly limit the depth of their dependency forest so that they can use a fixed number of buckets (presuming they use a bucketed approach at all—there are of course many other ways to architect a game loop).

Here's what a bucketed, phased, batched update loop might look like:

```
enum Bucket
{
    kBucketVehiclesPlatforms,
    kBucketCharacters,
    kBucketAttachedObjects,
    kBucketCount
};

void UpdateBucket(Bucket bucket)
{
    // ...

    for (each gameObject in bucket)
    {
        gameObject.PreAnimUpdate(dt);
    }

    g_animationEngine.CalculateIntermediatePoses
        (bucket, dt);

    for (each gameObject in bucket)
    {
        gameObject.PostAnimUpdate(dt);
    }

    g_ragdollSystem.ApplySkeletonsToRagDolls(bucket);
    g_physicsEngine.Simulate(bucket, dt); // ragdolls etc.
    g_collisionEngine.DetectAndResolveCollisions
        (bucket, dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons(bucket);
    g_animationEngine.FinalizePoseAndMatrixPalette
        (bucket);

    for (each gameObject in bucket)
    {
        gameObject.FinalUpdate(dt);
    }
}
```



```

        // ...
    }

    void RunGameLoop()
    {
        while (true)
        {
            // ...

            UpdateBucket (kBucketVehiclesAndPlatforms);
            UpdateBucket (kBucketCharacters);
            UpdateBucket (kBucketAttachedObjects);

            // ...

            g_renderingEngine.RenderSceneAndSwapBuffers();
        }
    }

```

In practice, things might be a bit more complex than this. For example, some engine subsystems like the physics engine might not support the concept of buckets, perhaps because they are third-party SDKs or because they cannot be practically updated in a bucketed manner. However, this bucketed update is essentially what we used at Naughty Dog to implement all of the games in the *Uncharted* series as well as *The Last of Us*. So it's a method that has proven to be practical and reasonably efficient.

### 16.6.3.3 Object State Inconsistencies and One-Frame-Off Lag

Let's revisit game object updating, but this time thinking in terms of each object's local notion of time. We said in Section 16.6 that the state of game object  $i$  at time  $t$  can be denoted by a state vector  $\mathbf{S}_i(t)$ . When we update a game object, we are converting its previous state vector  $\mathbf{S}_i(t_1)$  into a new current state vector  $\mathbf{S}_i(t_2)$  (where  $t_2 = t_1 + \Delta t$ ).

In theory, the states of all game objects are updated from time  $t_1$  to time  $t_2$  instantaneously and in parallel, as depicted in Figure 16.15. However, presuming that our game update loop is single-threaded, we actually update the objects one by one—we loop over each game object and call some kind of update function on each one in turn. If we were to stop the program halfway through this update loop, half of our game objects' states would have been updated to  $\mathbf{S}_i(t_2)$ , while the remaining half would still be in their previous states  $\mathbf{S}_i(t_1)$ . This implies that if we were to ask two of our game objects what the current time is during the update loop, they may or may not agree! What's more, depending on where exactly we interrupt the update loop, the objects

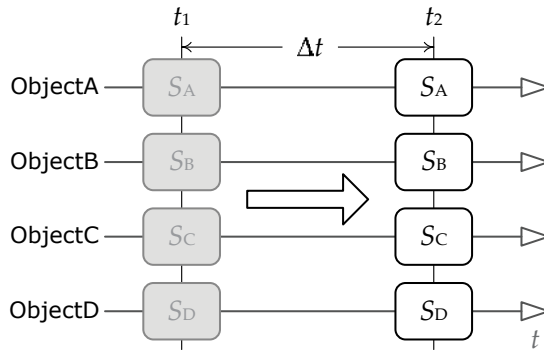


Figure 16.15. In theory, the states of all game objects are updated instantaneously and in parallel during each iteration of the game loop.

may all be in a partially updated state. For example, animation pose blending may have been run, but physics and collision resolution may not yet have been applied. This leads us to the following rule:

The states of all game objects are consistent *before* and *after* the update loop, but they may be inconsistent *during* it.

This is illustrated in Figure 16.16.

The inconsistency of game object states during the update loop is a major source of confusion and bugs, even among professionals within the game in-

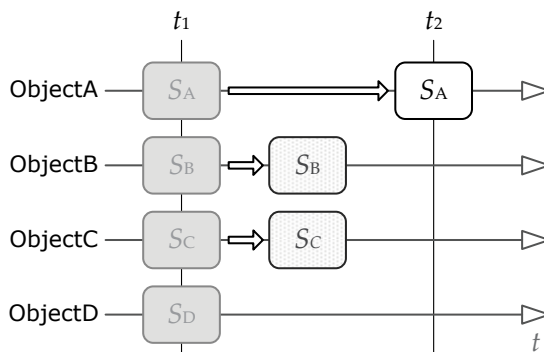


Figure 16.16. In practice, the states of the game objects are updated one by one. This means that at some arbitrary moment during the update loop, some objects will think the current time is  $t_2$  while others think it is still  $t_1$ . Some objects may be only partially updated, so their states will be internally inconsistent. In effect, the state of such an object lies at a point between  $t_1$  and  $t_2$ .

dustry. The problem rears its head most often when game objects query one another for state information during the update loop (which implies that there is a dependency between them). For example, if object B looks at the velocity of object A in order to determine its own velocity at time  $t$ , then the programmer must be clear about whether he or she wants to read the *previous* state of object A,  $S_A(t_1)$ , or the *new* state,  $S_A(t_2)$ . If the new state is needed but object A has not yet been updated, then we have an update order problem that can lead to a class of bugs known as *one-frame-off lags*. In this type of bug, the state of one object lags one frame behind the states of its peers, which manifests itself on-screen as a lack of synchronization between game objects.

#### 16.6.3.4 Object State Caching

As described above, one solution to this problem is to group the game objects into buckets (Section 16.6.3.2). One problem with a simple bucketed update approach is that it imposes somewhat arbitrary limitations on the way in which game objects are permitted to query one another for state information. If a game object A wants the *updated* state vector  $S_B(t_2)$  of another object B, then object B must reside in a *previously updated* bucket. Likewise, if object A wants the *previous* state vector  $S_B(t_1)$  of object B, then object B must reside in a *yet-to-be-updated* bucket. Object A should never ask for the state vector of an object within its own bucket, because, as we stated in the rule above, those state vectors may be only partially updated. Or at best, there's some uncertainty as to whether you're accessing the other object's state at time  $t_1$  or time  $t_2$ .

One way to improve consistency is to arrange for each game object to *cache* its previous state vector  $S_i(t_1)$  while it is calculating its new state vector  $S_i(t_2)$  rather than *overwriting* it in-place during its update. This has two immediate benefits. First, it allows any object to safely query the previous state vector of any other object without regard to update order. Second, it guarantees that a totally consistent state vector ( $S_i(t_1)$ ) will always be available, even during the update of the new state vector. To my knowledge there is no standard terminology for this technique, so I'll call it *state caching* for lack of a better name.

Another benefit of state caching is that we can linearly interpolate between the previous and next states in order to approximate the state of an object at any moment between these two points in time. The Havok physics engine maintains the previous and current state of every rigid body in the simulation for just this purpose.

The downside of state caching is that it consumes twice the memory of the update-in-place approach. It also only solves half the problem, because while

the previous states at time  $t_1$  are fully consistent, the new states at time  $t_2$  still suffer from potential inconsistency. Nonetheless, the technique can be useful when applied judiciously.

This technique is influenced strongly by the design principles of pure *functional programming* (see Section 16.9.2). In a pure functional programming language, all operations are performed by functions with a clear input and output, and no side-effects. All data is considered constant and immutable—rather than mutating an input datum in place, a brand new datum is always produced.

#### 16.6.3.5 Time-Stamping

One easy and low-cost way to improve the consistency of game object states is to time-stamp them. It is then a trivial matter to determine whether a game object's state vector corresponds to its configuration at a previous time or the current time. Any code that queries the state of another game object during the update loop can assert or explicitly check the time stamp to ensure that the proper state information is being obtained.

Time-stamping does not address the inconsistency of states during the update of a bucket. However, we can set a global or static variable to reflect which bucket is currently being updated. Presumably every game object “knows” in which bucket it resides. So we can check the bucket of a queried game object against the currently updating bucket and assert that they are not equal in order to guard against inconsistent state queries.

## 16.7 Applying Concurrency to Game Object Updates

In Chapter 4 we explored *hardware parallelism* and how to take advantage of the explicitly parallel computing hardware that has become the norm in recent gaming hardware. This is done via *concurrent programming* techniques. In Section 8.6, we introduced a number of approaches that allow a game engine to take advantage of the parallel processing resources. In this section, we'll take a look at ways in which concurrency and parallelism can be applied to the problem of updating the states of our game objects.

### 16.7.1 Concurrent Engine Subsystems

Clearly, the most performance-critical parts of our engine—such as rendering, animation, audio and physics—are the ones that will benefit most from parallel processing. So, whether or not our game object model is being updated in a

single thread or across multiple cores, it needs to be able to *interface* with low-level engine systems that are almost certainly multithreaded.

If our engine supports a general-purpose *job system* (see Section 8.6.4), we can use that job system to make our engine subsystems execute concurrently. In this scenario, each subsystem's frame update might be kicked off as a single job each frame. However, it's probably a better idea for each subsystem to kick off multiple jobs to perform its work each frame. For example, an animation system might kick off one job for each object in the game world that requires animation blending. Later in the frame, when the animation system performs its world matrix and skinning matrix computations, it could potentially employ a scatter/gather approach to divide this work across the available cores.

When making our low-level engine systems update concurrently, we need to ensure that their *interfaces* are *thread-safe*. We want to be sure that no external code can get into a data race situation, either in contention with other external clients, or in contention with the inner workings of the subsystem itself. This typically involves using locks on all external calls into each subsystem.

If our job system employs *user-level threads* (coroutines or fibers), then we will need to use spin locks to make our subsystems thread-safe. However, if our subsystems are updated by OS threads, then mutexes are also a viable option for this purpose.

If we can be certain that a particular engine subsystem only ever runs during one particular phase of the game loop, we may be able to employ *lock-not-needed assertions* rather than actually having to lock certain parts of the subsystem. See Section 4.9.7.5 for more information on these useful assertions.

Of course, another option is to attempt to use *lock-free* data structures to implement our engine subsystems critical (shared) data. Lock-free data structures are tricky to implement, and some data structures still have no known lock-free implementation. As such, if you opt for a lock-free approach, it's probably wise to restrict your efforts only to engine subsystems with the most stringent performance requirements.

## 16.7.2 Asynchronous Program Design

When interfacing with concurrent engine subsystems (for example from within a game object update), we must begin thinking *asynchronously*. When a time-consuming operation is to be performed, we should therefore avoid calling a *blocking function*—a function that does its work directly in the context of the calling thread, thereby blocking that thread or job until the work has been completed. Instead, whenever possible, large or expensive jobs should be requested by calling a *non-blocking function*—a function that sends the request

to be executed by another thread, core or processor and then immediately returns control to the calling function. The calling thread or job can proceed with other unrelated work, perhaps including updating other game objects, while we wait for the results of our request. Later in the same frame, or in next frame, we can pick up the results of the request and make use of them.

For example, a game might request that a ray be cast into the world in order to determine whether the player has line-of-sight to an enemy character. In a synchronous design, the ray cast would be done immediately in response to the request, and when the ray casting function returned, the results would be available, as shown below.

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r = castRay(playerPos, enemyPos);

    // Now process the results...
    if (r.hitSomething() && isEnemy(r.getHitObject()))
    {
        // Player can see the enemy.
        // ...
    }

    // ...
}
```

In an asynchronous design, a ray cast request would be made by calling a function that simply sets up and enqueues a ray cast job, and then returns immediately. The calling thread or job can continue doing other unrelated work while the job is being processed by another CPU or core. Later, once the ray cast job has been completed, the calling thread or job can pick up the results of the ray cast query and process them:

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r;
    requestRayCast(playerPos, enemyPos, &r);
```

```

// Do other unrelated work while we wait for the
// other CPU to perform the ray cast for us.

// ...

// OK, we can't do any more useful work. Wait for the
// results of our ray cast job. If the job is
// complete, this function will return immediately.
// Otherwise, the main thread will idle until the
// results are ready...
waitForRayCastResults(&r);

// Process results...
if (r.hitSomething() && isEnemy(r.getHitObject()))
{
    // Player can see the enemy.
    // ...
}

// ...
}

```

In many instances, asynchronous code can kick off a request on one frame, and pick up the results on the next. In this case, you may see code that looks like this:

```

RayCastResult r;
bool rayJobPending = false;

SomeGameObject::Update()
{
    // ...

    // Wait for the results of last frame's ray cast job.
    if (rayJobPending)
    {
        waitForRayCastResults(&r);

        // Process results...
        if (r.hitSomething() && isEnemy(r.getHitObject()))
        {
            // Player can see the enemy.
            // ...
        }
    }
}

```

```

    // Cast a new ray for next frame.
    rayJobPending = true;
    requestRayCast(playerPos, enemyPos, &r);

    // Do other work...
    // ...
}

```

### 16.7.2.1 When to Make Asynchronous Requests

One particularly tricky aspect of converting synchronous, unbatched code to use an asynchronous, batched approach is determining *when* during the game loop (a) to kick off the request and (b) to wait for and utilize the results. In doing this, it is often helpful to ask ourselves the following questions:

- *How early can we kick off this request?* The earlier we make the request, the more likely it is to be done when we actually need the results—and this maximizes CPU utilization by helping to ensure that the main thread is never idle waiting for an asynchronous request to complete. So for any given request, we should determine the earliest point during the frame at which we have enough information to kick it off, and kick it there.
- *How long can we wait before we need the results of this request?* Perhaps we can wait until later in the update loop to do the second half of an operation. Perhaps we can tolerate a one-frame lag and use last frame's results to update the object's state this frame. (Some subsystems like AI can tolerate even longer lag times because they update only every few seconds.) In many circumstances, code that uses the results of a request can in fact be deferred until later in the frame, given a little thought, some code refactoring, and possibly some additional caching of intermediate data.

### 16.7.3 Job Dependencies and Degree of Parallelism

In order to make the best use of a parallel computing platform, we'd like to keep all cores busy at all times. Presuming we're using a job system to parallelize our engine, each iteration of our game loop will be comprised of hundreds or even thousands of jobs running concurrently. However, *dependencies* between these jobs can lead to less-than-ideal utilization of the available cores. If job B takes as its input some data that is produced by job A, then job B cannot begin until job A is done. This creates a *dependency* between job B and job A.

The *degree of parallelism* (DOP) of a system, also known as its *degree of concurrency* (DOC), measures the theoretical maximum number of jobs that can



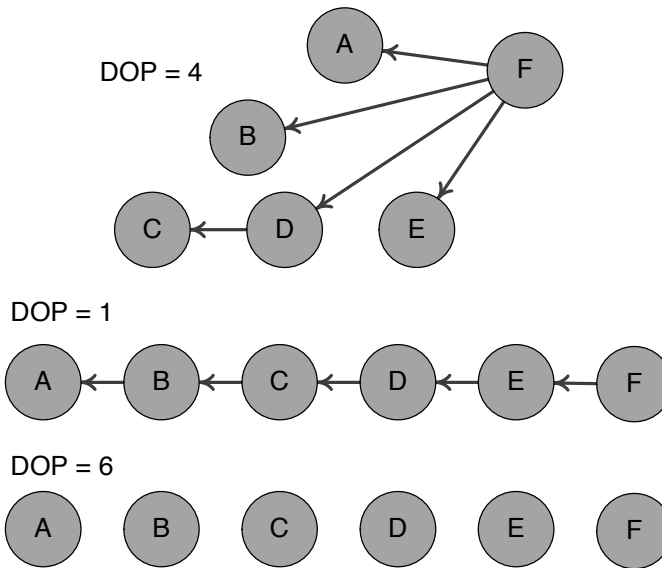


Figure 16.17. Three job dependency trees. The nodes of the tree are jobs, and the arrows represent dependencies between them. The number of leaves in such a tree indicates the system's degree of parallelism (DOP).

be running in parallel at any given moment in time. The degree of parallelism of a group of jobs can be determined by drawing a *dependency graph*. In such a graph, the jobs form the nodes of the tree, and the parent-child relationships represent dependencies. The number of leaves of the tree tells us the degree of parallelism of that collection of jobs. Figure 16.17 illustrates a few examples of job dependency trees and their corresponding degrees of parallelism.

In order to achieve full utilization of CPU resources in an explicitly parallel computer, we would like the DOP of our system to match or exceed the number of available cores. If the DOP of the software exactly equals the number of cores, we get maximum throughput. When the DOP of our system is higher than the number of cores, throughput is reduced because some jobs must run serially, but no core is idle. But when our system's DOP is less than the number of cores, some cores won't have any work to do.

Whenever a job is forced to wait for the jobs on which it depends to complete their tasks, a *synchronization point* (or "sync point" for short) is introduced into the system. Each sync point represents an opportunity for valuable CPU resources to be wasted while a job waits for its dependent jobs to complete their work. This is illustrated in Figure 16.18.

To maximize hardware utilization, we can attempt to increase the DOP of

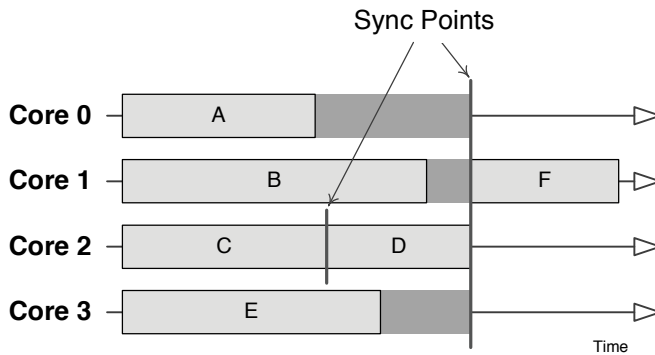


Figure 16.18. A synchronization point is introduced whenever one job is dependent upon the completion of one or more other jobs. Here, job D depends on job C, and job F depends on jobs A, B, D and E.

our system by reducing dependencies between jobs. We could also try to find some other unrelated work to do during idle periods. We can also reduce or eliminate the impact of a sync point by *deferring* it. For example, let's say that job D cannot commence its work until jobs A, B and C are done. If we try to schedule job D before A, B and C are all done, then it will obviously have to sit idle for some time. But if we defer job D so that it runs well after A, B and C are done, then we can be confident that D will never have to wait. This idea is illustrated in Figure 16.19. In his talk entitled "Diving Down the Concurrency Rabbit Hole" ([http://cellperformance.beyond3d.com/articles/public/concurrency\\_rabit\\_hole.pdf](http://cellperformance.beyond3d.com/articles/public/concurrency_rabit_hole.pdf)), Mike Acton says, "The secret of optimized concurrent design is *delay*." This is what he's talking about: Deferring sync points as a means of reducing or eliminating idle time in a concurrent system.

#### 16.7.4 Parallelizing the Game Object Model Itself

Game object models are notoriously difficult to parallelize for a few reasons. Game objects tend to be highly interdependent, and are typically dependent on numerous engine subsystems as well. Inter-object dependencies arise because game objects routinely communicate with one another and query one another's states during their updates. These interactions tend to occur multiple times during the update loop, and the pattern of communication can be unpredictable and highly sensitive to the inputs of the human player and the events that are occurring in the game world. This makes it difficult to process game object updates *concurrently* (using multiple threads or multiple CPU cores).

That being said, it certainly is possible to update a game object model con-

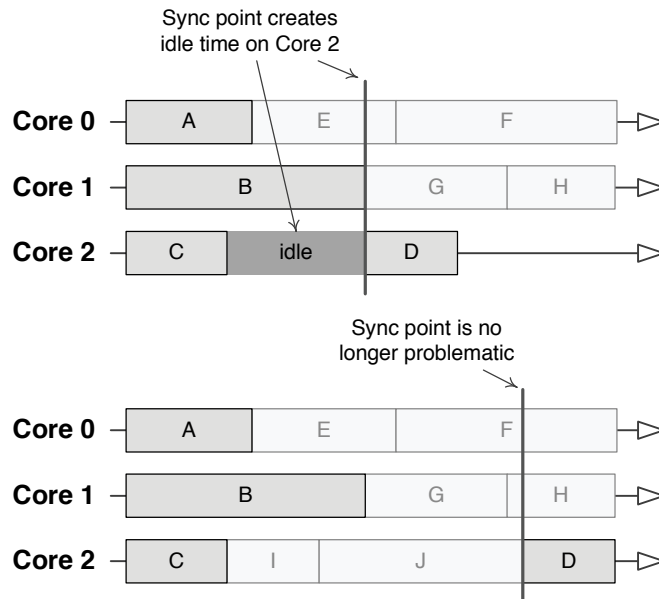


Figure 16.19. Job D depends on jobs A, B and C. Top: If we attempt to schedule job D immediately after job C on Core 2, the core will sit idle waiting for job B to complete. Bottom: If we delay job D's invocation until well after jobs A, B and C have completed, we free up Core 2 to run other jobs, thereby avoiding the idle time that had been caused by the sync point.

currently. As one example, Naughty Dog implemented a concurrent game object model when we ported our engine from PS3 to PS4 for *The Last of Us: Remastered*. In this section, we'll explore some of the problems one encounters when updating game objects concurrently, and we'll have a look at some of the solutions that Naughty Dog uses in our engine. Of course, these techniques are just one possible way to tackle the problem—other engines may use different approaches. And who knows? Perhaps *you* will invent a novel way of solving some of the vexing problems of concurrent game object model updates!

#### 16.7.4.1 Game Object Updates as Jobs

If our game engine has a job system, it will very likely be used to parallelize the various low-level subsystems in our engine, such as animation, audio, collision/physics, rendering, file I/O, and so on. So why not parallelize our game object updates by making them jobs, too? This is the approach used in the Naughty Dog engine. It works, but getting it to work correctly, and efficiently, is a non-trivial undertaking.

We discussed in Section 16.6.3.2 that, due to interdependencies between game objects, we need to control the order in which they update. One way to achieve this is to divide all of the objects in the game into  $N$  buckets, such that the objects in bucket  $B$  depend only on objects in buckets 0 through  $B - 1$ . If we take this approach, we could update all the game objects in each bucket by kicking them off as jobs, and let the job system schedule them across the available cores (and interspersed with whatever other jobs happen to be running at the time). This is roughly the technique employed in the Naughty Dog engine.

```
void UpdateBucket(int iBucket)
{
    job::Declaration aDecl[kMaxObjectsPerBucket];
    const int count = GetNumObjectsInBucket(iBucket);
    for (int jObject = 0; jObject < count; ++jObject)
    {
        job::Declaration& decl = aDecl[jObject];
        decl.m_pEntryPoint = UpdateGameObjectJob;
        decl.m_param = reinterpret_cast<uintptr_t>(
            GetGameObjectInBucket(iBucket, jObject));
    }
    job::KickJobsAndWait(aDecl, count);
}
```

Another way to deal with inter-object dependencies is to make them explicit. In this case, we would presumably have some way of declaring that game object A depends on game objects B, C and D, for example. These dependencies would form a simple *directed graph*. To update the game objects, we'd start by kicking off update jobs for all of the game objects with no dependencies on any other game object (i.e., those nodes of the dependency graph with no outgoing edges). As each job completes, we would walk to each dependent game object, and wait until all of its dependent objects' update jobs are complete. This process would repeat until the entire graph of game objects has been updated.

We can get into trouble with a graph of game object dependencies if the graph contains any *cycles*. A dependency cycle involving two or more game objects indicates that those objects cannot simply be updated in dependency order. To handle cycles, we either need to eliminate them by changing the way the objects interact (turning our graph into a *directed acyclic graph* or DAG), or we need to isolate these “clumps” of cyclically-dependent game objects and update each clump serially on a single core.

#### 16.7.4.2 Asynchronous Game Object Updates

We said in Section 16.7.1 that game object updates are usually done asynchronously. For example, rather than calling a *blocking function* to cast a ray, we would kick off an asynchronous *request* for a ray cast, and the collision subsystem would process this request at some future time during the frame. Later in the frame, or next frame, we would pick up the results of the ray cast, and act on them.

This approach continues to work well when the game objects themselves are also updating concurrently (across multiple cores). However, if our job system is based on *user-level threads* (coroutines or fibers), then *blocking calls* become a viable option, too. This works because a coroutine has the unique property of being able to *yield* (to another coroutine) and then continue where it left off at some later time (when another coroutine yields back to it). In a fiber-based job system (like the one used in the Naughty Dog engine), jobs aren't coroutines *per se*, but they do have this same property: A fiber-based job is able to “go to sleep” midway through its execution, and later to be “woken up” to continue execution where it left off.

Here's an example ray cast implemented using a blocking call, in a coroutine- or fiber-based job system:

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r = castRayAndWait(playerPos, enemyPos);

    // zzz...

    // Wake up when ray cast result is ready!

    // Now process the results...
    if (r.hitSomething() && isEnemy(r.getHitObject()))
    {
        // Player can see the enemy.
        // ...
    }

    // ...
}
```

Notice that this implementation looks nearly identical to the example of what *not to do* that we presented in Section 16.7.2! Thanks to user-level threads, the

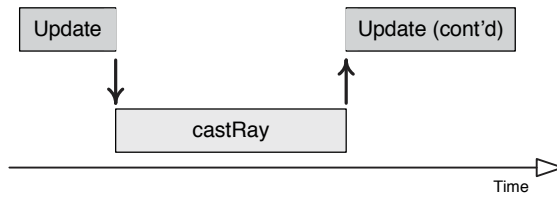


Figure 16.20. If our job system is implemented with user-level threads, a job can be interrupted part-way through its execution in order to perform an asynchronous operation, and resumed once that operation has been completed.

execution of our job can make use of blocking function calls after all! Figure 16.20 illustrates what is happening: The job has effectively been cleaved into two parts—the portion that runs before the blocking ray cast call, and the portion that runs after.

This mechanism is what allows us to implement job system functions like `WaitForCounter()` `KickJobsAndWait()`. These functions *block* their jobs, putting them to sleep and permitting other jobs to execute while they wait for the counter(s) in question to hit zero.

#### 16.7.4.3 Locking During Game Object Updates

Bucketed updates go a long way to solving the problems that arise due to inter-object dependencies. They ensure that the game objects are updated in the correct global order (e.g., the train car updates before the objects sitting on it). And they help with inter-object state queries. An object in bucket  $B$  can safely query the state of objects in buckets  $B - \Delta$ , and those in buckets  $B + \Delta$  (where  $\Delta > 0$ ), because in both cases we know that those objects won't be updating concurrently with the objects in bucket  $B$ . There is still a one-frame-off issue, however: If on frame  $N$ , an object in bucket  $B$  queries an object in bucket  $B - \Delta$ , it will see the state of that object on frame  $N$ ; however, if it queries an object in bucket  $B + \Delta$ , it will see the state of that object as it was last frame (on frame  $N - 1$ ) because it won't have updated yet.

So, with a bucketed updating system, we can safely access game objects in *other* buckets without needing any kind of lock. However, what about when game objects in the *same* bucket need to interact or query one another? Here, we are once again prone to concurrent race conditions, so we can't just do nothing and cross our fingers.

We might be tempted to introduce a single, global lock (mutex or spin lock) into the game object system. Every game object within one particular bucket could acquire this lock, perform its update (possibly interacting with other

game objects in the process), and then release the lock when it's done. This would certainly guarantee that inter-object communication would be free from data races. However, it would also have the highly undesirable effect of *serializing* the updates of all game objects within the bucket, reducing our “concurrent” game object update in effect to a single-threaded update! This occurs because the lock would prevent any two game objects from ever updating concurrently, even when those two objects don't interact in any way.

There are all sorts of other ways to tackle this problem. One approach we tried early on at Naughty Dog was to introduce a global locking system, but to only acquire the lock whenever a game object handle was *dereferenced* within a game object's update function. Game objects in our engine are referenced by handle rather than by raw pointers, to support memory defragmentation. So to get a raw pointer to a game object, one must first dereference its handle. This is a perfect opportunity to detect that one game object intends to interact with another. By only taking locks when interactions are actually likely to happen, we were able to recover some degree of concurrency. However, this locking system was complex, difficult to work with, and still led to inefficient use of the CPU cores during bucket updates.

#### 16.7.4.4 Object Snapshots

Upon analyzing the inter-dependencies between the game objects in a real game engine, we can make an observation: A large majority of the interactions between game objects during their updates are state queries. In other words, game object A reaches out and queries the current state of game objects B, C, and so on. When game object A does this, it really only needs to have access to a *read-only copy* of the states of these other game objects. It needn't interact with the objects themselves (which might or might not be concurrently updating at the moment of such interactions).

Given this state of affairs, it makes sense to arrange for each game object to provide a *snapshot* of its relevant state information—a read-only copy which can be queried, without locks or fear of data races, by any other game object in the system. Snapshots are really just an example of the *state caching* technique we described in Section 16.6.3.4. We used this approach at Naughty Dog for *The Last of Us: Remastered*, *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy*.

Here's how snapshots work in the Naughty Dog engine: At the start of each bucket update, we ask each game object to update its snapshot. These updates never query the state of other game objects, so they can be run concurrently without locks. Once all updates are complete, we kick off concurrent jobs to update the game objects' internal states. These too run concurrently.

Whenever a game object in bucket  $B$  needs to query the state of another object, it now has three options:

1. When querying an object in buckets  $B - \Delta$ , it can query it directly or query the object's snapshot.
2. When querying an object in bucket  $B$ , it queries the snapshot.
3. When querying an object in buckets  $B + \Delta$ , it again can query either the object itself, or its snapshot.

#### 16.7.4.5 Handling Inter-Object Mutation

Snapshots allow us to avoid locks when game objects *read* one another's game state. However, they do nothing to address the data races that can occur when one game object needs to *mutate* the state of another object in the same bucket. To handle these situations, Naughty Dog applied a combination of the following rules of thumb and techniques:

- Minimize inter-object mutation wherever possible.
- Inter-object mutation *between* buckets is safe, but
- inter-object mutation *within* a bucket must be handled carefully...
  - with locks,
  - or by requesting mutations by placing them onto a request queue, rather than applying the mutations immediately. The request queue is itself protected by a lock, and the handling of the requests in the queue is deferred until after the bucket update has completed.

Another option for inter-object mutation within a single bucket is to spawn a job in the *next* bucket whose job it is to synchronize these mutation operations. By deferring the action to a later bucket, we can be certain that the objects in question have completed their updated and will not be updating concurrently with our work. We did this at Naughty Dog to handle synchronized melee moves involving the player and an NPC.

#### 16.7.4.6 Future Improvements

The bucketed updating system we've described in this section isn't perfect by any means. Bucketed updates aren't as efficient as they could be, because each transition between buckets introduces a *sync point* into the game loop. At these sync points, some CPU cores may sit idle while we wait for all game objects in the bucket to complete their updates.